ПРАКТИЧЕСКОЕ ЗАДАНИЕ

по теме «Виды и инструменты автоматизации»

Студент: Назин. М.С.

Группа: ИСП.П 23.09.2

Дата: 28.10.2026

Вариант: чоткий

Результаты выполнения практической работы

1. Фактические HTTP-статусы и результаты проверок

Запрос	Метод	Ожидаемый результат	Фактический результат	Вывод
Create User	POST	200 OK, возвращен user_id, данные сохранены в переменные	**200 OK**. Тесты "User was created" и "User ID is returned" пройдены.	Успешно
User Login	POST	200 OK, возвращен session_token и user_id	**200 OK**. Тесты "Login is successful", "Session token is returned", "Username is returned" пройдены.	Успешно
Account summary	GET	200 OK, возвращен баланс и user_id	**200 OK**. Все 4 теста пройдены успешно.	Успешно
User Logout	GET	200 OK, сессия завершена	**200 OK**. Тест "Logout is successful" пройден.	Успешно
Account summary after logout	GET	401 Unauthorized или 403 Forbidden (доступ запрещен)	**401/403**. Тест "Access after logout is denied" пройден.	Успешно

2. Полный запуск коллекции (Collection Runner)

- Запросы выполнены строго в заданном порядке (через папки Setup и New user workflow).
- Передача данных между запросами работает корректно: user_id и token успешно передаются через переменные коллекции.

- Все автоматические проверки (Tests) выполнены без ошибок (0 failed).
- Скриншот окна Collection Runner приложен: 07_collection_runner.png.

3. Проверка доступа после Logout

После успешного выполнения запроса User Logout токен сессии становится недействительным. Повторный запрос Account summary after logout возвращает ошибку авторизации (статус 401 или 403), что подтверждает корректную работу механизма безопасности API. Скриншот: 06_account_after_logout.png.

4. Анализ результатов и баг-репорт

Дефект не подтвержден.

Обоснование вывода:

- **Фактический HTTP-статус:** Все запросы вернули ожидаемые статусы (200 для успешных операций, 401/403 для заблокированного доступа после выхода).
- **Содержание ответа:** Структура JSON-ответов соответствует документации (данные вложены в объект response).
- **Результаты автоматических проверок:** Все написанные тесты (проверка статусов, наличия user_id, balance, session_token и совпадения username) выполнены успешно.
- **Соответствие ожиданиям:** Поведение API полностью соответствует логике работы банковского приложения: данные передаются, сессия создается и корректно закрывается, доступ к защищенным данным без валидного токена блокируется.

5. Ответы на вопросы

Почему user_id устанавливается ещё раз после выполнения Login?
При создании пользователя (Create User) возвращается его уникальный идентификатор. При авторизации (Login) сервер также возвращает user_id вместе с session_token. Повторное сохранение user_id в переменную коллекции после Login гарантирует, что последующие запросы (например, Account summary) будут обращаться к данным именно того пользователя, чья сессия сейчас активна. Это защищает от рассинхронизации данных, если в скриптах отрабатывает несколько пользователей подряд.

Как изменится решение, если токен нельзя хранить в переменной коллекции?
Если хранение токена в переменной коллекции запрещено политиками безопасности (например, из-за риска утечки при экспорте коллекции или шаринге), решение необходимо изменить следующим образом:

1. **При локальном запуске (Postman):** Использовать тип переменных окружения (Environment) с пометкой *Secret* или встроенный *Postman Vault* (хранилище секретов).
2. **При запуске в CI/CD:** Секрет (токен) должен храниться в защищенных переменных CI-системы (например, GitHub Actions Secrets, GitLab CI Variables). При запуске коллекции через Newman токен передается в качестве аргумента

командной строки (--env-var "token=$TOKEN") или через файл окружения, который не попадает в репозиторий (добавлен в .gitignore).

3. **Почему нельзя хранить в репозитории/коллекции:** Коллекции и файлы окружения часто коммитятся в Git. Если сохранить токен в них, он попадет в историю репозитория, станет доступен всем, у кого есть доступ к коду, и его будет крайне сложно удалить отовсюду. Кроме того, это нарушает принцип наименьших привилегий (Least Privilege).